Revisiting Using the Results of Pre-Executed Instructions in Runahead Processors

Sonya R. Wolff, *Student Member, IEEE* and Ronald D. Barnes, *Member, IEEE*

Abstract—
Long-latency cache accesses cause significant performance-impacting delays for both in-order and out-of-order processor systems. To address these delays, runahead pre-execution has been shown to produce speedups by warming-up cache structures during stalls caused by long-latency memory accesses. While improving cache related performance, basic runahead approaches do not otherwise utilize results from accurately pre-executed instructions during normal operation. This simple model of execution is potentially inefficient and performance constraining. However, a previous study showed that exploiting the results of accurately pre-executed runahead instructions for out-of-order processors provide little performance improvement over simple re-execution. This work will show that, unlike out-of-order runahead architectures, the performance improvement from runahead result use for an in-order pipeline is more significant, on average, and in some situations provides dramatic performance improvements. For a set of SPEC CPU2006 benchmarks which experience performance improvement from basic runahead, the addition of result use to the pipeline provided an additional speedup of 1.14X (high – 1.48X) for an in-order processor model compared to only 1.05X (high – 1.16X) for an out-of-order one. When considering benchmarks with poor data cache locality, the average speedup increased to 1.21X for in-order compared to only 1.10X for out-of-order.

Index Terms—Runahead, Pre-Execution, Memory Wall.

1 Introduction

Runahead (RA) execution enables a processor to warmup cache memory structures during pipeline stalls caused by long-latency, cache-missing load instructions. Proposed for both in-order (IO) [1] and out-of-order (OO) [2] pipelines, RA approaches have been shown to improve performance with minimal hardware overhead. Basic RA pipelines re-execute all instructions encountered during RA mode. Using an OO model of RA execution, [3] showed that using RA executed results for instruction retirement provides little to no performance improvements over traditional RA operation. This work potentially discourages future research into alternative architectures that broadly exploit pre-execution results. While we confirm the conclusions of [3] for OO pipelines, utilizing pre-executed instruction results by eliminating re-execution of RA instructions shows much more significant performance promise in IO-style pipelines. This paper explores the execution differences between in-order runahead (IORA) and out-of-order runahead (OORA) systems that enable IO systems to experience significant speedups from using pre-executed RA results. Simulation results, presented in Section 4, demonstrate these performance differences between in-order runahead with result use (IORARU) and out-of-order runahead with result use(OORARU) systems and the impact these results might have on an IORA pipeline design.

2 Runahead Execution

Over many processor generations, the impact of the "memory wall" [4] has resulted in an ever increasing number of cycles required to service cache-missing memory requests. Runahead pre-execution attempts to mitigate the impact of these requests by warming up cache structures during pipeline stalls caused by long-latency, cache-missing load requests. Cache warm-up is achieved by placing the pipeline into RA mode while the load request is being processed by the cache. Once in RA mode, instructions that depend on long-latency data are marked "dirty". Since RA mode disables the architectural register state's update, the pipeline can continue instruction execution during RA mode when it would otherwise be stalled. By allowing instruction execution to proceed, additional, independent cache-missing loads are likely to be encountered. By initiating these loads, RA mode pre-loads the cache hierarchy with blocks that will be needed once normal operation resumes. Since caches preserve the warm-up effects of RA operation, significant performance improvement can be achieved without using the results of pre-executed RA instructions. However, several pre-execution approaches have shown non-negligible performance improvements by using the correctly calculated RA instruction results [5]–[11] in apparent conflict with [3].

Processor pipelines fetch a program's individual instructions in the sequence those instructions appear in memory. Ideally, this instruction sequence would be optimized to produce the minimum number of stalls. Often, this is not possible because of dynamic behavior including program branching and variable latency instructions (especially memory accesses) [12]. OO processors handle these variable latency and suboptimal instruction sequences by placing the sequentially fetched instructions into instruction queues (IQs). The order of execution is then determined by the status of an instruction's dependencies. Once an instruction's source registers are available, that instruction is available for execution regardless of its position in the program's sequential ordering. As long as ready instructions are available, long latency instruction execution will not result in a pipeline stall. Only when all instructions waiting in the IQs depend on cache-missing load data will those loads produce pipeline stalls. RA execution further masks the impact of cache-missing loads by entering RA mode anytime a cache-missing load instruction blocks the in-program-order retirement from the reorder buffer. Basic RA is supported by taking a checkpoint of system state at the beginning of RA mode and restoring this state once the cache request completes.

An IO processor execute instructions in program order. Thus, a multiple-cycle-latency instruction can produce pipeline a stall whenever the compiler fails to schedule for an instruction's full latency. Even with aggressive compiler scheduling and large

Date of publication 16 Jul. 2013; date of current version 31 Dec. 2014.
Digital Object Identifier 10.1109/L-CA.2013.21

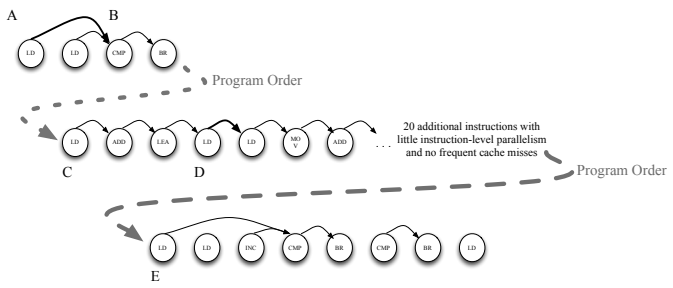

Fig. 1: Code segment from *astar*

TABLE 1: Processor Configuration

Pipeline A	in-order	8-wide fetch and retire
Pipeline B	out-of-order	8-wide issue ; 64-entry LSQ; 128-entry ROB; 128-entry RS; 256 physical registers
Common Setup	Execution Core	8 ALU, Latency: IMUL(8), FMUL(4), FCTV(4), FCMP(4), FADD(4), FDIV(16), other(1)
	I-Cache	64KB, 4-Way, 2 Cycle, 4 loads per cycle
	L1 Cache	64KB, 4-Way, 2 Cycle, Write-Through, 4 ld/st per cycle, LRU Replacement
	Unified L2 Cache	1MB, 32-Way, 10 cycle, Write-Back, 1 ld/st per cycle, LRU Replacement
	Memory	minimum 500 latency, 32 banks, 32B wide, 4:1 split trans. core-to-memory at 4:1 freq. ratio

caches, IO processors suffer significantly from the performance impacts of cache misses [13]. Since the stall for a cache-missing load instruction occurs soon after execution, an IORA pipeline enters RA mode whenever a long-latency cache-missing load instruction executes in normal mode. Performance can potentially be improved using RA results during normal mode. Long latency instructions can use their precomputed results in a single cycle, and an instruction's dependences are broken by using the results from the already executed RA instruction.

Inherent differences between IO and OO processors account for the contrast in performance improvements achievable by using pre-executed instruction results during normal mode. Since IO pipelines cannot naturally mask program-order dependancies, the IORARU pipeline benefits from the latency masking provided by using RA results. An OORARU pipeline cannot benefit from this masking to the same extent since it already effectively mask the latency of most instructions through OO execution.

In addition to these behavioral differences, the power requirement to support IORARU execution is likely significantly less than needed to support OO execution. An OO pipeline's IQ require content-addressable memories or many-ported wired-OR resource matrices to support waking up instructions once their data-flow dependences are met. These hardware structures are more complex than the FIFO-based IQ and result queue need to enable IORARU's pre-executed result use for instruction re-issue during normal mode's program order execution. A speculative copy of the register file is needed for use during RA mode (with or without re-use). Replicating storage for each register in a register file has been shown to add only a small additional area or latency [14] and register file checkpoints have previously been proposed to support branch misprediction recovery [15] and hardware transactional memory [16]. The FIFO-based IQ and replicated register file approach is significantly simpler and less-power consuming than the register alias table, large physical register file, and reorder buffer needed to support OO execution. In [7], it was determined that the structures needed to support OO execution consume as much as 10X the power as the corresponding structures in an IORARU implementation.

3 Runahead with Result Use: Example

Figure 1 shows a subsection of code from the *astar* benchmark. The first load operation (**A**) causes most of the RA mode entries for both inputs, *astar.river* and *aster.lakes*. This load is quickly followed by a consumer (**B**) which stalls the pipeline under normal IO operation. The RA-causing load operation (**A**) is followed by three instructions, two of which, are on a dependent chain. The subsequent four instructions (**C**) are independent of both previous load operations. The new instruction chain results in a second cache-missing load (**D**). The 23 instructions after the load (**D**) form a dependent chain and would be deferred during RA mode. In a worst-case single issue pipeline, it would require only 30 cycles to reach the next independent instruction string (**E**) since the 20 instructions not shown would all be deferred. Since RA targets long-latency loads, the instructions at **E** should be reached during RA mode and are also independent of the **A** and **D** loads. The instruction chain starting at E normally executes cleanly during RA mode.

In both IO and OO pre-execution models, the relatively short sequence (**C**) and the longer sequence (**E**) will execute and produce correct results. IORARU pipelines will exploit both of these pre-executed sequences, accelerating execution by skipping the already pre-executed instructions. However, the benefit during OO execution is not nearly as great. Since normal execution will restart with the initial load (**A**), the successfully pre-executed sequence (**C**) will result in a slight benefit from reuse. Unfortunately, the much longer pre-computed sequence (**E**) does not provide similar speedups for a standard OO pipeline. Instead, once normal mode resumes, the chain (**E**) will be processed simultaneously with the instructions deferred as a result of the load miss (**D**). Thus, the sequence (**E**) overlaps with the dependent chain (**D**) through normal OO execution producing no performance benefit from OORARU beyond that provided by basic OORA.

The two *astar* benchmark display a moderate to average performance improvement from RU; however, both versions of *soplex* (*ref* and *pds*) experience significant speedups from RU. The above example is similar to a single RA episode for *soplex*, but the percentage of execution time in RA mode for *astar* and *soplex* are extremely different. While *astar* is in RA mode 16 to 38% of the overall execution time, *soplex* is in RA mode 46 to 71% of the time. A more complete analysis of this behavior is discussed in the next section; however, overall RA utilization is the major difference in performance improvements between these two benchmarks.

4 Simulation Results

Simulation experiments were performed using Soonergy [17], a cycle-accurate architectural simulator. Processor configurations used are shown in Table 1. These models attempt to approximate the configuration outlined in [3]. While this configuration represents an extremely aggressive pipelining design, [18] and results in Fig. 5 and Table 3 demonstrate that the performance benefits discussed are consistent with narrower issue width. To demonstrate the performance difference from result use for a RA pipeline, six different execution models were examined: IO, IORA, IORARU, OO, OORA and OORARU.

The programs simulated were complied from the set of C/C++ language SPEC CPU2006 benchmarks compatible with Microsoft Visual Studio 2010 on Microsoft Windows 7. Simulated runs consist of 250 million x86 instructions with an

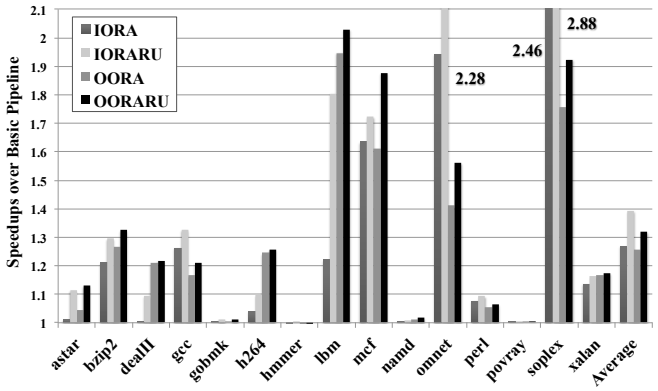

Fig. 2: Speedups over Baseline for each Runahead Type

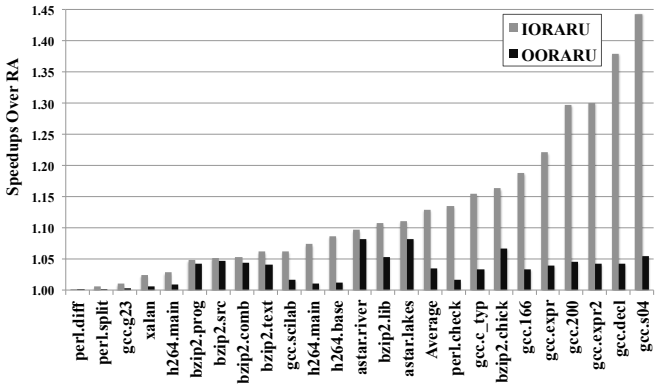

Fig. 3: Speedups for Runahead with Result Use: Medium

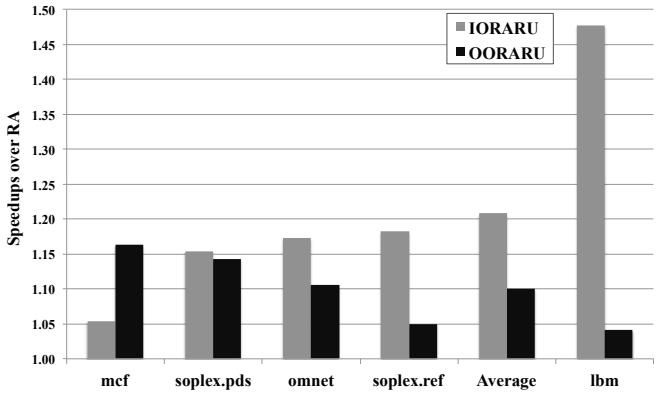

Fig. 4: Speedups for Runahead with Result Use: High

additional 25 million instruction warm-up period. This code segment represent a statistically significant portion of the program's execution selected using the technique in [19]. The SPEC CPU2006 benchmark suite provides multiple reference inputs for some benchmarks. For those with multiple inputs, all reference inputs were simulated. For brevity, a single result is sometimes reported for the multiple inputs. This single reported value is the average for each input weighted by the number of x86 instructions executed during a full program run with that input.

To compare RARU with the traditional RA approach, speedups values were calculated for each model (RA and RARU) over the corresponding basic pipeline (IO and OO). Fig. 2 shows that, while the impact from incorporating basic RA into the IO and OO pipelines is similar (average speedup: 1.27X vs. 1.26X), the speedup achieved for an IORARU pipeline is noticeably higher than for OORARU (average speedup: 1.40X vs. 1.32X). As can be seen in Fig. 2, the benchmarks can also be categorized into distinct groups based on RA improvement: low (*gobmk*, *hmmer*, *namd*, and *povray*), medium (*astar*, *bzip2*, *dealII*, *gcc*, *h264*, *perl*, and *xalan*), and high (*lbm*, *mcf*, *omnet*, and *soplex*). These three groups benefit from RA to a significantly varying degree. Benchmarks in the low performance group experience, on average, no speedup from any RA approach. The high performance group experiences speedups well over 1.69X in comparison to IO for all benchmarks. These speedup variations are largely explained by the number of long-latency cache misses which result in a varying number of entries into RA mode across all three groups.

TABLE 2: Entries into Runahead Mode (in thousands)

Groups	IORA	OORA	IORARU	OORARU
Low	8.91	44.09	9.73	44.38
Medium	143.36	115.39	167.57	116.22
High	2083.24	1269.93	2042.13	1218.35

For each of the four RA approaches, Table 2 shows the average number of times benchmarks in each group enter RA mode. Not unexpectedly, the low group, which benefits little from RA, rarely enters RA mode, whereas the medium and high performance groups utilize RA operation much more frequently. Frequent entries into RA mode enable the medium and high groups to reap greater benefits from the RA produced pre-warmed cache. Since the low performance group spends little execution time in RA mode, benchmarks in this category execute few instructions during RA operation. Hence, the benefit provided by RU is also minimal. Because the lack of improvement from RU is a direct result of poor RA utilization, these benchmarks are not considered further.

The performance impact of RA operation is significantly different for the medium and high groups. To clearly show the improvements possible from the addition of RU to an RA pipeline, separate figures were created for the RARU speedups over the basic RA operation, Fig. 3 (Medium) and Fig. 4 (High). These two figures represent the improvement achieved by adding RU to a RA pipeline and do not include the speedup achieved by RA itself. With the exception of *mcf*, the speedup for IORARU is higher than for OORARU for all benchmarks.

Mcf's lower performance on the IORARU pipeline is the result of only 48.7% of RA instructions being independent of a cache-miss from either the original RA-inducing instruction or a subsequent cache-missing load request. Because OO is able to rearrange the instructions' order of execution, the OORARU pipeline can tolerate longer secondary cache accesses and 69.8% of the OORARU pipeline's instructions execute independently during RA mode. Thus, the OORARU pipeline increases its potential to accelerate normal mode execution and shows a higher speedup for *mcf* than IORARU pipeline. Note that IORARU approaches that make multiple passes over instructions during RA [7], [9] could similarly tolerate these secondary misses, but results presented here are limited to to a single RA pass.

For the medium performance group, there is a large difference in speedup from results use with IORARU achieving a respectable 1.13X speedup over IORA, compared to much less significant 1.03X speedup for OORARU. While OORARU does have a significant speedup for the high performance group (an average of 1.10X), it is still significantly less than the average 1.21X speedup of the IORARU system. Addition-

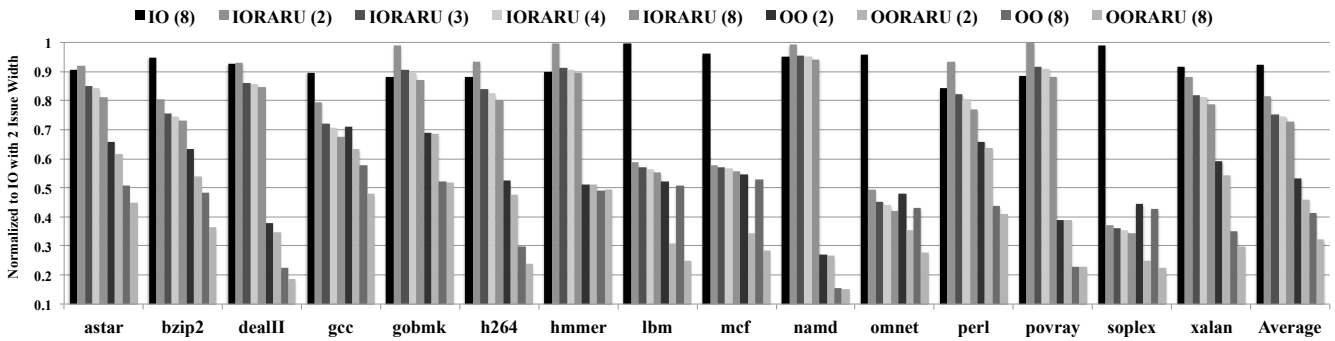

Fig. 5: Weighted Execution Time Normalized to a Two Issue IO Pipeline

ally, between the medium and high groups, nine different applications achieve speedups for IORARU greater than the single best OORARU speedup (*mcf*'s 1.16X). These higher IORARU speedups clearly demonstrate that an IO pipeline experiences significantly more performance improvement from the addition of result use to RA than an OO system.

TABLE 3: Weighted Execution Time (units: billions of cycles)

	Issue Width			
	2	3	4	8
IO	1.166	1.133	1.128	1.111
IORA	0.829	0.795	0.790	0.776
IORARU	0.763	0.724	0.716	0.700
OO	0.599	0.559	0.544	0.526
OORA	0.468	0.405	0.381	0.361
OORARU	0.436	0.385	0.353	0.333

The addition of RARU to an IO pipeline provides performance improvements approaching or even surpassing benefits provided by a basic OO pipeline. Fig. 5 graphs the weighted execution cycles for various issue widths for all simulated execution models. The difference in performance between the OO and the IORARU models is small for many of the programs (*gcc*, *lbm*, *mcf*, *omnet* and *perl*) and *soplex* has better performance under IORARU. These numbers are even more significant considering the baseline OO model is an extremely aggressive design. As the average weighted executions times in Table 3 show, the significant improvements from result use on IO relative to OO hold even for more feasible issue widths than considered in [3].

5 Conclusion

In [3], it was clearly demonstrated that for the OO processor model, using pre-executed results from RA instructions provided little performance improvements–likely discouraging future work in this area. While RU does provide minimal benefit for OORA execution, IO processor systems experience significantly greater speedup from RU. For some memory-dominated benchmarks, IORARU execution times decrease to near or shorter than OO pipelines due to the IORARU pipelines' ability to both tolerate long-latency cache misses and to exploit the results of non-memory pre-executed instructions. IORARU processor models also provide a way to improve the performance of an IO processor while potentially avoiding the power and complexity of a full OO implementation. This makes IORARU attractive for asymmetric multi-core processors in which memory-level-parallelism-targeted processors are coupled with more general purpose cores [20]. Since IORARU implementations have been shown to use less power than OO pipelines [7], the introduction of IORARU in a multi-core processor would allow high program throughput on relatively low power pipelines without the negative performance impacts typically seen with IO pipelines.

References

[1] J. Dundas and T. Mudge, "Improving data cache performance by pre-executing instructions under a cache miss," in *Proc. of the 11th Int'l. Conf. on Supercomp.*, 1997, pp. 68–75.
[2] O. Mutlu *et al.*, "Runahead execution: An effective alternative to large instruction windows," *IEEE Micro*, vol. 23, no. 6, pp. 20–25, 2003.
[3] O. Mutlu *et al.*, "On reusing the results of pre-executed instructions in a runahead execution processor," *IEEE Comput. Archit. Lett.*, vol. 4, no. 1, pp. 2–5, 2005.
[4] W. A. Wulf and S. A. McKee, "Hitting the memory wall: Implications of the obvious," *ACM SIGARCH Comput. Arch. News*, vol. 23, no. 1, pp. 20–24, 1995.
[5] R. D. Barnes *et al.*, "Beating in-order stalls with 'flea-flicker' two-pass pipelining," *IEEE Trans. Comput.*, vol. 55, pp. 18–33, 2006.
[6] H. Zhou, "Dual-core execution: Building a highly scalable single-thread instruction window," in *Proc. of the 14th Int'l. Conf. on Parallel Arch. and Compiler Techniques*, 2005, pp. 231–242.
[7] R. D. Barnes *et al.*, "Tolerating cache-miss latency with multipass pipelines," *IEEE Micro*, vol. 26, no. 1, pp. 40–47, 2006.
[8] S. T. Srinivasan *et al.*, "Continual flow pipelines," *ACM SIGARCH Comput. Arch. News*, vol. 32, no. 5, pp. 107–119, 2004.
[9] A. Hilton *et al.*, "iCFP: Tolerating all-level cache misses in in-order processors," in *Proc. of IEEE 15th Int'l. Sym. on High Perf. Comput. Arch.*, 2009, pp. 431–442.
[10] S. Chaudhry *et al.*, "Rock: A high-performance SPARC CMT processor," *IEEE Micro*, vol. 29, no. 2, pp. 6–16, 2009.
[11] S. R. Wolff and R. D. Barnes, "Re-examining instruction reuse in pre-execution approaches," 9th Annual Workshop on Duplicating, Deconstruction and Debunking Held in San Jose, CA, June 2011.
[12] P. P. Chang *et al.*, "Comparing static and dynamic code scheduling for multiple-instruction-issue processors," in *Proc. of the 24th Ann. Int'l. Sym. on Microarch.*, 1991, pp. 25–33.
[13] J. W. Sias *et al.*, "Itanium performance insights from the IMPACT compiler," in *Hot Chips 13*, 2001.
[14] E. S. Fetzer *et al.*, "The multi-threaded, parity-protected 128-word register files on a dual-core Itanium-family processor," in *IEEE Int'l. Solid-State Circuits Conf. Digest of Techn. Papers*, vol. 1, 2005, pp. 382–383, 605.
[15] W. W. Hwu and Y. N. Patt, "Checkpoint repair for out-of-order processors," in *Proc. of the 14th Int'l. Symp. on Comp. Arch.*, 1987, pp. 18–26.
[16] M. Herlihy and J. E. B. Moss, "Transactional memory: Architectural support for lock-free data structures," in *Proc. of the 20th Int'l. Symp. on Comp. Arch.*, 1993, pp. 289–300.
[17] N. Freeman, "Soonergy: A pluggable, cycle-accurate computer architecture simulator," Master's thesis, The University of Oklahoma, May 2011.
[18] S. R. Wolff, "Pre-execution: An elegant approach to the memory wall," Master's thesis, The University of Oklahoma, July 2011.
[19] T. Sherwood *et al.*, "Automatically characterizing large scale program behavior," in *Proc. of the 10th Int'l. Conf. on Arch. Sup. for Prog. Lang. and OS*, 2002, pp. 45–57.
[20] G. Patsilaras *et al.*, "Efficiently exploiting memory level parallelism on asymmetric coupled cores in the dark silicon era," *ACM Trans. on Arch. and Code Opt.*, vol. 8, no. 4, pp. 28:1–28:21, 2012.